Closed-Loop Threat-Informed Remediation of Cloud-Native Kubernetes Security Misconfigurations

Brian Mendonca and Vijay K. Madisetti, *Fellow, IEEE*

Abstract—Containerized applications depend on Kubernetes YAML manifests that configure images, permissions, and storage; small configuration errors can introduce outages or security gaps. Existing systems can detect, mutate, or generate repairs, but expose different and often narrower evidence for accepting a candidate patch. This paper studies `k8s-auto-fix`, a closed-loop remediation system that subjects deterministic and optional LLM candidates to an explicit Kubernetes-specific acceptance contract before review or application. In a fixture-seeded Kind/Kubernetes 1.34.0 replay, all 1,000 verifier-accepted patches passed both server-side dry-run and live apply checks. On a 15,718-detection offline run, deterministic rules plus safety checks accepted 13,589 of 13,656 attempted patches (99.51%; auto-fix rate 0.8646 over all detections; median patch ops 8). In a deterministic replay of a historical 1,259-item supported queue, static risk-priority ordering reduces top-50 P95 wait from 147.2 h to 7.8 h under a uniform 10-minute service-time fallback. An optional LLM mode reaches 88.52% acceptance on a matched 5,000-detection-record corpus. The released artifact contains the data and scripts used for these results.

Index Terms—Kubernetes, Static analysis, API-level validation, Admission control, Server-side dry-run, YAML, Pod Security, JSON Patch, Policy Enforcement, Kyverno, OPA Gatekeeper, Auto-fix, CI/CD, Exploit-catalog risk, Guidance retrieval, Risk-based scheduling

1 Introduction

Kubernetes is now a common substrate for cloud services, yet its declarative configuration model remains prone to security misconfigurations [33], [39]. A manifest is the text file that tells Kubernetes what to run, with which permissions, and on what resources. When these files are edited by hand, a stray `privileged: true`, a floating `:latest` image tag, or a missing `runAsNonRoot` line can quietly open a security gap or break a deployment.

Problem. Static analyzers detect misconfigurations; admission engines can validate or mutate resources, including existing resources; and recent LLM systems generate candidate repairs [19], [20], [27]. Their acceptance evidence, however, commonly stops at a subset of schema enforcement, scanner re-checks, or admission outcomes. We study a narrower question: can heterogeneous candidate patches be evaluated under one explicit contract that combines trigger-policy re-check, RFC 6902 applicability, Kubernetes server-side admission, and cross-policy safety invariants? This boundary matters because LLM reasoning over vulnerabilities remains unreliable without external checks [32].

Approach. We present `k8s-auto-fix`, a closed-loop *detect* → *propose* → *verify* → *prioritize* pipeline whose proposer defaults to deterministic rules and admits optional LLM-generated patches only behind the same guardrails. The design is *threat-informed* in two ways. First, the verifier records four gates—policy re-check, JSON Patch application, server-side API dry-run, and no-new-violation safety—and each campaign identifies which were required; live campaigns require all four. Second, the scheduler orders accepted work by a transparent static score built from policy risk, historical verifier pass priors, configured service-time priors, and configured exploit-catalog-style boosts; live vulnerability-feed joins are planned enrichment rather than required for the reported results.

Contributions. This paper makes the following contributions:

- A proposer-independent four-gate acceptance contract for Kubernetes repair candidates: trigger-policy re-check, JSON Patch application validation, Kubernetes API admission through server-side dry-run, and universal safety invariants. The released records identify the verifier configuration used by each campaign; the live replay exercises all four gates.
- An artifact-backed evaluation of verifier-gated repair, including a matched static replay of risk-ordered versus FIFO processing under an explicit service-time fallback.
- A reproducibility bundle for deterministic and LLM-backed modes on offline corpora and a fixture-seeded live cluster, with summary tables regenerated by `make reproducible-report` and per-claim command scope enumerated in ARTIFACTS.md.

Organization. Section 2 defines the task, acceptance boundary, and threat model. Section 3 presents the closed-loop

- *B. Mendonca is with the College of Computing, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: brian.mendonca6@gmail.com).*
- *V. K. Madisetti is with the School of Cybersecurity and Privacy, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: vkm@gatech.edu).*
- *Corresponding author: Vijay K. Madisetti.*

design, including the verifier and scheduler. Section 4 describes the implementation and artifacts. Section 5 evaluates acceptance, latency, risk reduction, and patch size. Section 6 reports verifier ablations and failure taxonomies. Section 7 compares against prior systems, Section 8 states limits, and Section 9 concludes.

2 Task and System Model

2.1 Model and Acceptance Predicate

We model a manifest as a JSON/YAML document m and a detection as a tuple $d = (m, \pi, v)$, where π is the violated policy identifier and v is the violation evidence emitted by the detector. The proposer produces a JSON Patch δ (RFC 6902 [9]), and applying it yields the patched manifest $m' = \delta(m)$. A detection is *resolved* when m' satisfies the acceptance predicate $\mathrm{Acc}(m', \pi) = \mathrm{policy}(m', \pi) \wedge \mathrm{patch}(m, \delta) \wedge \mathrm{dryrun}(m') \wedge \mathrm{safe}(m')$, i.e. the conjunction of the four verifier gates defined in Section 3. The scheduler ranks resolved items using policy risk R_i, a historical verifier success prior p_i, a configured service-time prior $\mathbb{E}[t_i]$, optional queue age wait_i, and a configured KEV-style boost kev_i. The reported replay initializes every item with zero age and zero exploration input, and all policy service-time priors fall back to 10 minutes; consequently, it evaluates static risk-priority ordering rather than online learning or an aging intervention.

Detector disagreement and false positives. The detector can take the union of `kube-linter` and policy-engine findings, but default verification uses explicit assertions for the triggering policy; an optional `-enable-rescan` hook can re-run external detectors. We therefore do not claim that every accepted record was re-scanned by both engines. Every deterministic proposer fix is guarded by JSON Pointer existence checks, so a finding that does not correspond to a real field produces no edit rather than a spurious change, and the policy re-check rejects any patch that does not clear the asserted violation. A detection the proposer cannot resolve is routed to human review.

Attempts and retry budget. An *attempt* is one proposer→verifier cycle for a single detection: the proposer emits a candidate patch and the verifier evaluates $\mathrm{Acc}(m', \pi)$. If an attempt fails, the failing gate and error trace are fed back and the proposer may retry, up to a configurable budget `max_attempts` (default 3, set in `configs/run.yaml`). After the budget is exhausted the item is dropped from automated handling and routed to review. The released `data/policy_metrics.json` stores historical policy pass priors and configured scheduler inputs; it is not evidence of an online bandit update loop.

2.2 Threat Model

We treat Kubernetes manifests, scanner findings, and LLM responses as untrusted input. Trusted components include the detector/verifier binaries, the scheduler, and the per-cluster fixtures under infra/fixtures/; these run inside the CI environment we control and write the artifacts cited throughout Section 5. The adversary may supply malicious YAML, poison retriever context passed to the LLM backend, or craft fixtures that cause the Kubernetes API server to reject dry-run requests. We do not claim to detect prompt injection. Instead, the system limits its consequences by treating model output as untrusted and rejecting candidates that violate the checked gates; semantically malicious but gate-passing patches remain subject to human review. Compromised detector binaries, forged audit logs, and malicious container-image supply chains are out of scope, as is scheduler-input poisoning.

2.3 Threats and Mitigations

The reproducibility bundle (`make reproducible-report`) regenerates an artifact-side acceptance table from JSON for cross-checking the overlapping rows of Table 4. Semantic regression checks now block Grok-generated patches that remove containers or volumes, and fixtures under infra/fixtures/ seed role-based access control (RBAC) and NetworkPolicy gaps before verification. We threat-modeled malicious or placeholder manifests: the retriever limits prompt context, the verifier enforces policy and campaign-required API admission, and the scheduler surfaces only configured-verifier passes. Residual risks—primarily infrastructure assumptions and LLM hallucinations—are characterized in the failure taxonomies (Tables 5 and 6). A privileged-DaemonSet case study in the artifact bundle shows the guardrails preserving required host integrations while removing privilege escalation paths.

Secret hygiene is enforced end-to-end: the proposer replaces secret-like environment values with `secretKeyRef` references, sanitizes generated names, and documents the enforced invariants in docs/security_considerations.md.

3 System Design

We implement the closed loop *Detector* → *Proposer* → *Verifier* → *Scheduler* shown in Figure 1. Detectors produce structured JSON findings; the proposer applies rule-based guards (with optional LLM backends) to emit minimal JSON Patches; the verifier enforces policy re-checks, patch-application validation, `kubectl apply -dry-run=server` API admission, and safety invariants; and the scheduler orders accepted work using a transparent risk-priority score. Each stage persists artifacts (detections, patches, verified outcomes, queue scores), enabling reproducible evaluation (Section 5).

3.1 Pipeline and Verifier Gates

The workflow consists of four stages: the **Detector** ingests a Kubernetes manifest and uses both `kube-linter` and a policy engine (Kyverno/OPA) to identify violations, taking the union of all findings; the **Proposer** takes the manifest and violation data and generates a JSON Patch, defaulting to deterministic rules for the covered policy families (image tags, privilege, capabilities, non-root execution, read-only root filesystems, host namespace/mount controls, seccomp, and resource requests/limits) while guarding each operation with JSON Pointer existence checks and enforcing minimality/idempotence via `tests/test_patch_minimality.py`; the **Verifier** applies the patch to a copy of the manifest and subjects it to

the verification gates described below, recording evidence in data/verified.json; and the **Budget-aware Retry** loop uses a configurable retry budget (max_attempts in configs/run.yaml, default 3) so the proposer can re-attempt if verification fails, logging the error trace for inspection.

The verifier implements four gates. The **Policy Re-check** re-evaluates the patched manifest with the same policy logic that triggered the violation, using explicit assertions for each covered policy (e.g., no_latest_tag, no_privileged, drop_capabilities, drop_cap_sys_admin, run_as_non_root, read_only_root_fs, no_host_* flags, no_allow_privilege_escalation, enforce_seccomp, set_requests_limits); the detector hook for re-scanning is available via -enable-rescan, and is not part of every recorded acceptance. Non-allowlisted hostPath volumes can be rewritten to emptyDir: {} by guardrails; because that rewrite can change storage semantics, it is marked for human review unless workload requirements are known. The **Patch-application Validation** gate applies the JSON Patch via jsonpatch and rejects malformed paths or operations. The **Server-side Dry-run/API Admission** gate uses kubectl apply -dry-run=server to ask the Kubernetes API server to validate the changed object, marking failures as not accepted and persisting CLI output. The **No-New-Violations Safety** gate enforces universal assertions: it blocks privileged: true; rejects dangerous capabilities.add entries; flags empty capabilities.drop lists when capabilities are present; requires non-empty container images; and restricts host mounts to approved paths (/var/run/secrets/kubernetes.io/serviceaccount, /var/lib/kubelet/pods, /etc/ssl/certs). Live campaigns require all four gates. Historical offline campaigns can run with the API gate optional, so their acceptance rates mean that the recorded campaign configuration passed and are not presented as API-admissibility rates.

Scheduler example. In a static queue with a privileged container, a ':latest' image, and a lower-risk resource-limit item, the privileged item receives the highest score and is served first. The implementation can add elapsed queue age when items arrive at different times, but the reported snapshot initializes all ages equally and does not evaluate whether aging prevents starvation under continuing arrivals.

3.2 End-to-End Walkthrough on Test Manifests

To make the closed-loop pipeline concrete, we trace two purpose-built manifests from the repository's test suite through detection, patching, verification, and scheduling.

Case 1: Remediating a Privileged Pod with a :latest Image Tag

The first smoke-test example shows a common but high-risk pattern: a privileged container using a floating tag.

```
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: app
    image: acme/api:latest
    securityContext:
      privileged: true
      allowPrivilegeEscalation: true
      capabilities: { add: ["SYS_ADMIN", "NET_ADMIN"] }
```

Walkthrough. We scan the file, draft a small idempotent fix, validate JSON Patch application, and run policy, server-dry-run/API-admission, and safety gates before queueing it. "Accepted" means the configured gates passed, not a guarantee of runtime workload correctness.

1. Detect (Union): The detector reports no_privileged, drop_capabilities, run_as_non_root, and no_latest_tag in a structured detection record.

2. Propose (Rules Engine): The proposer emits a minimal JSON Patch. The pinned tag (acme/api:1.42.0) is illustrative: the deterministic fixer requires an operator-configured replacement tag or digest for no_latest_tag and otherwise routes the item to review.

```
[
 {"op":"replace","path":"/spec/containers/0/
    securityContext/privileged","value":false},
 {"op":"replace","path":"/spec/containers/0/
    securityContext/allowPrivilegeEscalation","value":
    false},
 {"op":"remove","path":"/spec/containers/0/securityContext
    /capabilities/add"},
 {"op":"add","path":"/spec/containers/0/securityContext/
    capabilities/drop","value":["ALL"]},
 {"op":"add","path":"/spec/containers/0/securityContext/
    runAsNonRoot","value":true},
 {"op":"replace","path":"/spec/containers/0/image","value
    ":"acme/api:1.42.0"}
]
```

3. Verify (Four Gates): The verifier validates that the JSON Patch applies to an in-memory copy, confirms the original policies are cleared, checks the universal safety invariants, and succeeds under kubectl apply -dry-run=server in a fixture-seeded Kind cluster. The successful outcome is recorded in the verifier log.

4. Schedule (risk-aware priority): Because the item is high risk, historically fixable, and quick to verify, it receives a high score and moves earlier in the queue.

Case 2: Hardening a Worker Pod with a hostPath Mount

```
spec:
  containers:
  - name: worker
    securityContext: { readOnlyRootFilesystem: false }
    resources: {}
  volumes:
  - name: host
    hostPath: { path: "/var/run/docker.sock" }
```

The second smoke-test example targets a writable root filesystem, a dangerous hostPath, and missing resource requests/limits.

1–2. Detect and propose: The detector flags read_only_root_fs, no_host_path, and set_requests_limits; the rules engine hardens the filesystem, proposes replacing the disallowed hostPath with emptyDir, and adds resource quotas:

```
[
 {"op":"replace","path":"/spec/containers/0/
    securityContext/readOnlyRootFilesystem","value":true},

 {"op":"remove","path":"/spec/volumes/0/hostPath"},
 {"op":"add","path":"/spec/volumes/0/emptyDir","value
    ":{}},
 {"op":"add","path":"/spec/containers/0/resources","value
    ":{"requests":{"cpu":"100m","memory":"128Mi"},"limits
    ":{"cpu":"500m","memory":"256Mi"}}}
]
```

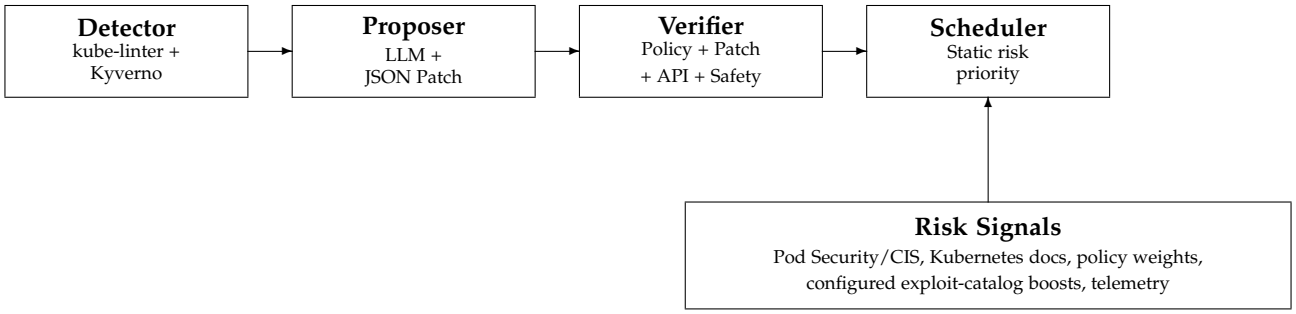


Figure 1. Closed-loop architecture. The verifier implements policy re-check, patch-application validation, server-side dry-run/API admission, and no-new-violation gates; the live replay requires all four, while historical offline artifacts record their campaign-specific configuration.

3–4. Verify and schedule: The verifier confirms that the JSON Patch applies, the `no_host_path` assertion is gone, the universal safety checks pass, and server dry-run accepts the object. Allowlisted host mounts would be preserved, and the policy re-check discipline is the same mechanism that catches the ablation escapes in Section 5. Because replacing a host mount can change workload semantics, this accepted patch is review-required unless the workload's storage requirement is known. The item receives a moderate score and is scheduled behind higher-risk work.

Comparison with existing approaches. Kyverno mutation operates during admission and through background `mutateExisting` policies, and depends on cluster fixtures and policy/controller context for complex hardening. GenKubeSec localizes and explains issues but leaves remediation and validation manual, and LLMSecConfig generates LLM repairs with scanner checks but lacks our combined patch-application, server-side dry-run, and universal-safety boundary.

Acceptance boundary and scheduling. The boundary matrix shows that weaker acceptance predicates admit 312–551 records rejected by the corresponding full-verifier snapshots on the shared 5k records (Table 7); live-cluster replay recorded 100% server-side dry-run and live apply acceptance on the fixture-seeded subset. In the matched finite queue replay, static risk priority reduces top-50 P95 wait from 147.2 h to 7.8 h under a uniform 10-minute service-time fallback.

Table 1 makes the lifecycle gap explicit: comparable systems can detect, mutate, or suggest repairs, but they differ sharply on the evidence an operator receives before trusting a patch. We use "safety invariants" below for universal no-new-violation gates beyond confirming that one scanner finding disappeared.

3.3 Risk Scoring

We rank accepted fixes by policy risk and configured verifier-cost priors. The evaluated replay uses policy weights, configured KEV-style flags, historical policy pass rates, and a uniform 10-minute service-time fallback; public CVE, Exploit Prediction Scoring System (EPSS), and Known Exploited Vulnerabilities (KEV) feed joins are planned enrichment.

The scheduler consumes per-policy records that store historical pass probability, configured service-time prior, KEV-style flag, and baseline risk. A calibration pass records corpus-level ΔR and residual risk (Table 3); the corresponding artifact paths and regeneration commands are listed in `ARTIFACTS.md`. Future iterations will enrich each queue item with container-image vulnerability joins via `Trivy` and `Grype` [16], [17], Common Vulnerability Scoring System (CVSS)/EPSS feeds [13], [15], and the Cybersecurity and Infrastructure Security Agency (CISA) KEV catalog [14].

3.4 Guidance Refresh and Retrieval Hooks

The artifact curates guidance from Kubernetes Pod Security Standards, Center for Internet Security (CIS) benchmarks, and Kyverno sources [1], [2], [26]. LLM-backed proposer modes can retrieve these snippets at prompt time, and the roadmap extends this into a full retrieval-augmented generation loop: chunk guidance with metadata (policy family, resource kind, field path, image→vulnerability), cache recent verifier failures, and retrieve targeted passages when retries occur. This keeps the prompt budget bounded while grounding fixes in up-to-date hardening language.

3.5 Static Risk-Priority Scheduler

Overview. The released scheduler is a deterministic priority queue. It scores each accepted item from explicit inputs and sorts scores in descending order. **Notation.** R_i denotes policy risk units for item i; p_i is its historical verifier pass prior; $\mathbb{E}[t_i]$ is a configured service-time prior; wait_i is optional elapsed queue age; α weights that age; kev_i is the configured KEV-style bonus when a policy is flagged (otherwise 0); and ε is a small positive floor preventing division by zero.

$$S_i = \frac{R_i \cdot p_i}{\max(\varepsilon, \mathbb{E}[t_i])} + \alpha\,\text{wait}_i + \text{kev}_i \tag{1}$$

The risk construction is explicit. Each detection maps to a policy identifier and receives static weight w_{policy}. The configured KEV-style bonus remains the separate kev_i term in Equation 1; it is not an external KEV or EPSS feed value in the evaluated artifact. Formally,

$$R_i = w_{\text{policy}}, \qquad \text{kev}_i = \kappa \cdot \mathbf{1}_{\text{configured KEV}},$$

where κ is configured by the caller. The released replay uses $\alpha = 1.0$ and $\kappa = 1.0$; the age term is inert because every initial wait is zero. We also report $\Delta R_i = R_i - R_i^{\text{post}}$ for accepted risk-mapped records, summing that subset to produce Table 3.

Table 1
Lifecycle contract for remediation tools: what evidence exists before an operator can trust a patch.

Lifecycle question	k8s-auto-fix (this work)	Kyverno	Kubecurity	LLMSecConfig
When does it act?	Post-hoc on stored manifests and replayed live resources	Admission create/update and background mutation of existing resources	Offline LLM correction plus deterministic schema repair	Offline iterative LLM repair
What patch artifact exists?	Minimal RFC 6902 JSON Patch plus verifier evidence	Mutation-policy result	Corrected YAML	Corrected Kubernetes configuration
What accepts the patch?	Campaign-recorded policy, patch-application, optional/required-by-campaign API-admission, and safety evidence	Admission/background-controller outcome	Official Kubernetes JSON-schema enforcement and misconfiguration evaluation	YAML validation + static-analysis re-scan
How is work ordered?	Matched static risk-priority replay	Controller processing; no evaluated risk queue	None	None

The current release does not compute an upper-confidence bound, update priors after dequeue, or evaluate staggered arrivals. Although the persistent queue can derive elapsed age at selection time, every item in the reported batch snapshot has the same enqueue time and a zero initial age; the additive age term therefore cannot change relative order. Section 5 consequently compares the static score $R_i p_i / \mathbb{E}[t_i] + \text{kev}_i$ against FIFO on identical accepted records. Online learning and open-arrival fairness remain future work.

4 Implementation

The implementation follows the design above but keeps the default reproducible path independent of external LLM APIs. A three-stage command-line pipeline records detections, candidate patches, verifier outcomes, and scheduler scores as JSON/CSV artifacts; optional LLM-backed modes use the same verifier boundary. The released artifact maps each reported claim to source files and regeneration commands in `ARTIFACTS.md`, while the main text focuses on the interfaces that affect the evaluation.

Reproducing the results. The pipeline runs as `make detect`, `make propose`, and `make verify`; `make reproducible-report` regenerates summary tables from checked-in JSON/CSV outputs. Full-corpus regeneration (15,000+ detections) takes roughly 20–30 minutes; the dependency snapshot is `data/repro/environment.json`.

Scalability and integration. The pipeline sustains millisecond-scale proposer and sub-second verifier latency on the supported corpus (Table 4); the scheduler replays thousands of queue items without recomputing detections. Rules, LLM, and replay modes all cross the same acceptance boundary.

4.1 Detection Records and Test Evidence

Each detector record carries the fields {`id`, `manifest_path`, `manifest_yaml`, `policy_id`, `violation_text`}; the embedded `manifest_yaml` decouples downstream stages from the filesystem, so the proposer and verifier operate on the record alone. A representative record and its full schema ship with the released artifact (`data/detections.json`).

The test suite (`make test`) covers detector contracts, proposer guards, verifier gates, scheduler ordering, and patch idempotence, with generated patch-minimality checks gated behind `make artifact-test`. Property-based guard tests run over hundreds of randomized manifests per run, confirming that the per-policy patchers add the expected hardening (dropping dangerous capabilities, denying privilege escalation, enforcing `RuntimeDefault` seccomp, preferring non-privileged defaults) and remain idempotent without widening the universal verifier gate beyond the invariants of Section 3.

4.2 Dataset and Configuration

Two deliberately vulnerable manifests (`001.yaml`, `002.yaml`) are retained for smoke tests. The matched Grok-5k corpus contains 5,000 detection records drawn from 1,021 unique YAML documents, while the supported corpus contains 1,264 detection records drawn from 402 unique YAML documents after policy normalization. Multiple detections may therefore share one manifest, and record-level acceptance rates are not manifest-level estimates. `configs/run.yaml` remains the source of truth for proposer mode, retry budgets, and API endpoints. The opt-in Grok/xAI rerun configuration uses `grok-4.3` against the xAI chat-completions endpoint with temperature 0, seed 1337, a 60 s timeout, and two retries per call; switching out of deterministic rules mode requires the corresponding external credentials. The supplemental appendix documents the ArtifactHub mining pipeline and corpus hashes [29], and `ARTIFACTS.md` maps each table, figure, and headline claim to its data source.

The privileged-DaemonSet case study follows the same artifact trail. For a Cilium-style manifest, the proposer preserves required host mounts while changing `privileged` and `allowPrivilegeEscalation` to false, dropping all Linux capabilities by default, and enforcing `RuntimeDefault` seccomp. The example is retained in `docs/privileged_daemonsets.md`; the paper uses it only to illustrate how verifier gates preserve required host integrations while blocking privilege-escalation paths.

Cross-cluster replay used 200 manifests per provider-targeted environment and the same four verifier gates plus fixtures. The EKS-targeted slice succeeded on 198/200 manifests with 198/198 accepted patches; the GKE and AKS slices each succeeded on 200/200 with 200/200 accepted patches. We report these as replay outputs (see Table 10); broader multi-provider generalization remains future work. Per-provider `summary.csv` and `results.json` files live under data/cross_cluster/{eks,gke,aks}/, with collection steps in docs/cross_cluster_replay.md.

5 Evaluation

5.1 Research Questions

We organize the evaluation around four research questions. The design makes a specific bet—that verification, not the generator, should be the acceptance boundary—and each question probes one consequence of that bet: whether the loop accepts useful fixes (RQ1), whether static risk-priority ordering improves high-risk queue position relative to FIFO (RQ2), what fairness conclusions the bounded replay can and cannot support (RQ3), and whether accepted patches are small enough to review (RQ4). RQ1 and RQ4 evaluate threat removal under the verifier boundary; RQ2 and RQ3 evaluate threat prioritization under the risk-aware queue.

RQ1 (Robustness): does the closed loop accept fixes across corpora and modes? *Rationale:* a remediation loop is useful only if a large fraction of attempted patches survive the verifier configuration recorded for each campaign. *Finding:* the checked-in artifacts record 88.52% acceptance on the Grok-5k detection-record sweep, 100.00% on the supported 1,264-record rules corpus, and 13,589/13,656 (99.51%) accepted on the full 15,718-detection run under deterministic rules plus guardrails (auto-fix rate 0.8646 over detections; median 8 operations). Only the fixture-seeded live replay is used to claim server-side dry-run and live-apply success.

RQ2 (Scheduling effectiveness): does static risk-priority ordering reduce the wait for dangerous items? *Rationale:* operators triage a backlog, so the metric of interest is how early the highest-risk items are served when throughput is held fixed. *Finding:* on the same historical supported queue, the static score reduces top-50 P95 wait from 147.2 h under FIFO to 7.8 h (18.8×) under the replay's uniform 10-minute service-time fallback.

RQ3 (Fairness): what fairness conclusion does the replay support? *Rationale:* a static priority queue can defer low-risk work under continuing arrivals. *Finding:* both permutations drain the same finite 1,259-item queue, so total throughput and the all-item wait distribution are unchanged by ordering. Because all initial ages are zero and no new items arrive, this replay neither demonstrates starvation under open arrivals nor isolates an aging effect.

RQ4 (Patch quality): are the accepted patches small and stable? *Rationale:* a patch that an operator must hand-audit is only reviewable if it is compact and does not drift under reapplication. *Finding:* generated JSON Patches have median 8 operations on the full 15,718-detection corpus and 5–8 on smaller curated slices; rule-patcher tests check idempotence under repeated application.

5.2 Experimental Setup and Results

Unless explicitly noted, results derive from the deterministic `rules` pipeline. Table 2 states the unit behind each headline number, Table 4 consolidates acceptance with regenerated latency where available, and Table 10 distinguishes completed campaigns, deterministic replays, and planned work. The API-backed Grok mode is benchmarked as an opt-in configuration (4,426 / 5,000 accepted) because it requires external credentials and funded access. `ARTIFACTS.md` maps each table, figure, and headline claim to its source files and regeneration commands.

Detector evidence. The detector stage wraps `kube-linter` and policy-engine findings; we do not claim a novel detector. A synthetic nine-policy hold-out confirms that the wrapper emits the expected finding on each purpose-built fixture. Because each policy has exactly one fixture, this is a regression-coverage check rather than a real-world accuracy estimate. The live replay validates API-admission and checked-invariant compliance for the items the detector flags—every accepted patch passed server-side dry-run and live apply checks—but it does not establish full semantic safety or detector recall on uncurated manifests.

ArtifactHub structural labels. To avoid scoring against detector-derived labels, we use a separately implemented structural oracle for the 69 checked-in ArtifactHub manifests. It parses YAML fields against configured policy semantics, including absent non-root, read-only-root-filesystem, resource-limit, and capability-drop settings, before scoring detector output. Against these labels, the wrapper records 143 true positives, 19 false positives, and three false negatives: structural-agreement precision 0.883 (95% bootstrap CI 0.843–0.916), structural-agreement recall 0.979 (0.962–1.000), structural-agreement F1 0.929 (0.902–0.948), and structural-agreement weighted F1 0.944 (0.932–0.960). Residual disagreements are concentrated: 16/19 false positives are capability-related policy-boundary cases, two are service-selector cases, one is a Job TTL case, and all three false negatives are service-account references in generated job/cronjob manifests. We therefore treat this as structural-oracle agreement and error-analysis evidence, not a broad detector benchmark; labels, residual errors, and protocol files are mapped in `ARTIFACTS.md`.

Acceptance by corpus and mode. The evaluation campaigns span deterministic and LLM-backed modes. Rules mode repairs 1,264/1,264 supported detection records (100%) with 29 ms median proposer latency and 242 ms median verifier latency (P95 517.8 ms). The archived Grok/xAI 5k cache delivers 4,426/5,000 accepted record-level remediations (88.52%) with median JSON Patch ops 9; telemetry records 4.38M input, 0.69M visible completion, and 11.40M total tokens including xAI-reported reasoning tokens, allowing cost to be recomputed from current pricing [11]. On the full rules+guardrails run (15,718 detections), the current raw verifier records yield 13,589/13,656 accepted patches (99.51%; auto-fix rate 0.8646 over detections) with median patch ops 8. Figure 3 uses only the matched 5,000-record rules and Grok snapshots; it does not mix these rows with the full corpus.

Latency and failures. To ground deployability, we instrumented Grok/xAI proposer and verifier traces from a separate archived 200-detection-record replay (56 unique manifest paths). The proposer shows 5.10 s median end-to-end latency (P95 16.9 s), while the verifier adds 89.5 ms median latency (P95 369.3 ms). Failure causes remain dominated by dry-run contract mismatches and legacy StatefulSets; Table 5 summarises the top categories and ties each mitigation to a regression class. These latency medians are not a full-5k latency estimate; full 5k acceptance and token telemetry are reported separately.

The failure taxonomy (Table 5) shows that the largest Grok dry-run failure category (65 cases) comes from the

Table 2
Metric denominators. Each headline number is computed over a specific unit and corpus; we state these explicitly to avoid ambiguity across acceptance, auto-fix, and risk metrics.

Dataset	Unit	Count	Mode	What the metric measures
Live replay	manifests	1,000	rules + seeded fixtures	server-side dry-run and live apply acceptance
Supported corpus	detection records	1,264	rules	record-level acceptance (402 unique YAML documents)
Supported risk set	detection records	1,278	rules + risk scheduler	queue replay; 1,264 records are risk-mapped
Full corpus (detections)	detections	15,718	rules + guardrails	total detections found
patched subset	patches	13,656	rules + guardrails	patches attempted
accepted subset	patches	13,589	rules + guardrails	verifier-accepted (99.51% of attempted)
Grok/xAI slice	detection records	1,313	LLM (opt-in)	paired rules-vs-Grok record acceptance (100.00%)
Grok-5k	detection records	5,000	LLM (opt-in)	record-level LLM patch acceptance (1,021 unique YAML documents)

Definitions. "Acceptance" is computed over *attempted patches on detection records*; "auto-fix rate" (0.8646) is computed over *all detections* and means verifier-accepted without manual patch editing, not automatic production deployment; live-replay "success" means both server-side dry-run and live apply passed for the fixture-seeded manifest. Multiple detection records can share one YAML document, so record-level acceptance and Wilson intervals are descriptive and do not adjust for within-manifest clustering. Risk removal is computed only where a policy-risk mapping exists.

Kubernetes API refusing to return an existing object, often for custom resources needing elevated role-based access control (RBAC); 20 cases come from stale core/v1 resource lookups, and the long tail is dominated by invalid StatefulSet/CronJob specifications. These counts shaped the mitigations used uniformly in rules and Grok modes: seed custom-resource and RBAC fixtures, pre-flight StatefulSets for missing `volumeMounts`, and route immutable-field dry-run errors to human review.

Live replay and guardrails. Live-cluster replay on a stratified 1,000-manifest subset in the fixture-seeded Kind/Kubernetes 1.34.0 evaluation environment achieves 100.0% server-side dry-run and live apply acceptance (1,000/1,000) with full alignment between the two checks on this subset. The verifier seeds bespoke service accounts and injects benign placeholder images where manifests omit them. Guardrail importance is quantified by a 5k boundary matrix computed from checked-in verifier-record snapshots: weaker policy-only or admission-style boundaries admit 312 rules-mode records and 551 Grok/xAI policy-only records that the corresponding full-verifier snapshots reject (Table 7). Because these inputs are archived verifier records, the matrix should be read as snapshot-derived guardrail evidence rather than a live implementation rerun.

Scheduling and risk removal. The matched replay contains 1,259 accepted detections from a historical 1,278-detection supported queue. Static risk-priority ordering places the top-50 risk items at median rank 25.5 (P95 rank 48), versus median rank 303.5 (P95 rank 884) under FIFO. With the same uniform 10-minute service-time fallback for every queued item, this corresponds to top-50 median/P95 waits of 4.1/7.8 h under risk priority and 50.4/147.2 h under FIFO. Because ordering is the only changed variable, both schedules drain the same queue in 209.8 h at 6 items/hour and have the same all-item wait distribution. Figure 2 reports the top-50 comparison. Separately, risk calibration covers the 1,264 records with policy-risk mappings, of which 1,247 were accepted, and removes 55,935/56,990 configured units (98.15%); 14 queue records, including 12 accepted `no_host_path` records, are excluded from those risk totals (Table 3).

This finite replay establishes a prioritization effect, not a starvation guarantee. Measuring starvation requires an open-arrival workload and staggered queue ages; those conditions are absent here.

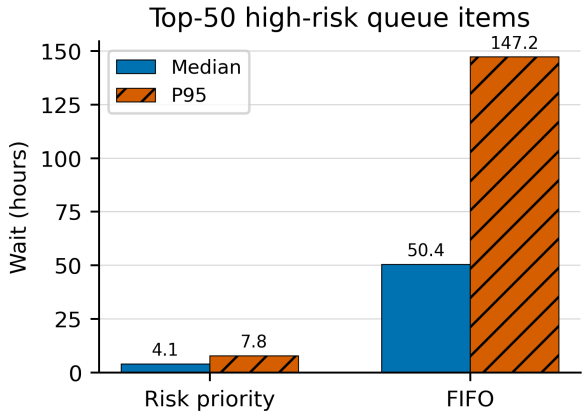


Figure 2. Median and P95 wait for the top-50 highest-risk items in the matched 1,259-item historical supported-queue replay. Both schedules use identical inputs and a uniform 10-minute service-time fallback; all initial ages and exploration inputs are zero.

Baseline comparison. Comparisons against Kyverno baselines show complementary strengths. The live-cluster Kyverno mutate run accepts 364/381 detections (95.54%) once patched manifests pass our verifier, and the mutating webhook exceeds 98% on fixture-compatible overlapping policies while remaining lower for policies that require more context. Our deterministic full-corpus rules+guardrails run reaches 99.51% over attempted patches, and the 19-sample verifier-ablation study reaches 78.9% under full gates while catching four policy escapes. These rows use different corpora and are coverage context, not a leaderboard.

Reading Tables 3–4. ΔR is risk removed by accepted fixes; residual is mapped risk left; $\Delta R/t$ is risk removed per minute of configured work; P95 is the 95th-percentile time; and auto-fix counts verifier-accepted patches that required no manual patch editing. The row is a mapped subset, not the full 1,278-record queue: 14 records lack a calibration mapping. Within the subset, a privileged pod carries 85 units, missing `runAsNonRoot` carries 70, and a floating `:latest` tag carries 50. Removing 55,935 of 56,990 mapped units retires 98.15% of that configured-risk total. Scheduler

Table 3
Risk calibration for the policy-risk-mapped supported subset. $\Delta R/t$ divides risk removed by summed 10-minute priors over accepted mapped records.

Dataset	Det.	Accepted	ΔR	Residual	ΔR/R	ΔR /t
Risk-mapped supported subset	1,264	1,247	55,935	1,055	98.15%	4.49

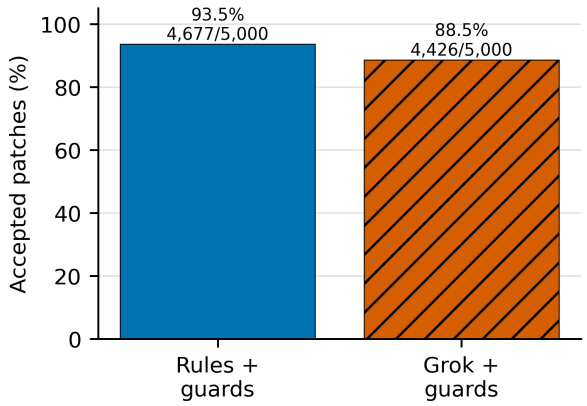


Figure 3. Matched 5,000-record verifier snapshots: rules+guardrails accept 4,677/5,000 (93.54%) and Grok+rule-guardrails accept 4,426/5,000 (88.52%). Both rows use byte-identical detection input; they are archival snapshot evidence rather than current proposer/verifier reruns.

replay inputs are constructed separately and provide a 10-minute fallback for all 1,259 queued items.

Detailed per-record deltas between rules and Grok/xAI on the 1,313-record slice are documented in the project artifact docs/ablation_rules_vs_grok.md. The operator survey instrument is drafted in docs/operator_survey.md; it documents the planned human-in-the-loop rotation listed in Table 10.

6 Ablations and Failure Taxonomy

Table 6 compares two historical verifier snapshots. The cohorts differ, so the table describes failure composition and does not identify fixture seeding as the sole cause of the difference. Counts are rejected records carrying each normalized category; records may carry more than one category. The full-rules snapshot has 67 rejected records among 13,656, and the supported snapshot has 19 among 1,278. A separate current normalized supported run accepts 1,264/1,264 and is not merged into these historical columns.

6.1 Scheduler and Verifier Ablations

Scheduler replay. The released comparison is regenerated from 1,259 accepted records in a historical 1,278-detection supported queue. FIFO preserves detection order; risk priority sorts the same records by Equation 1. For the top-50 risk items, FIFO yields median/P95 ranks 303.5/884 and waits 50.4/147.2 h, while risk priority yields ranks 25.5/48 and waits 4.1/7.8 h. Every expected-time input is the configured 10-minute fallback, every initial age is zero, and every exploration input is zero. Thus the replay supports only a static ordering comparison; it does not support a UCB, exploration, aging, or operator-acceptance effect.

Acceptance-boundary ablation. Table 7 quantifies the acceptance boundary on matched 5k rules+guardrails and Grok+rule-guardrails verified-record snapshots. The matrix holds candidate-patch records fixed and weakens the acceptance predicate: accepting every emitted patch, accepting only records that clear the original scanner/policy assertion, accepting records that clear the policy and recorded API-admission flag, and finally applying the corresponding full-verifier snapshot. The API-admission column uses the stored server-side dry-run flag from the verifier record; in the artifact this field is named `ok_schema` for historical reasons. The weaker boundaries accept more records, but they also admit records the full-verifier snapshot rejects. On rules-5k, policy-only and admission-style boundaries would accept 312 additional historical snapshot rejects; on Grok-5k, policy-only acceptance would admit 551 dry-run or safety rejects. These are snapshot-derived results, not current verifier reruns. Figure 3 uses those same two matched snapshots.

Terminology. An *ablation* weakens the recorded acceptance predicate to measure its effect; an *escape* in Table 7 is a record that would be accepted by that weaker predicate but was rejected by the corresponding full-verifier snapshot.

Domain-invariant enforcement vs. generic acceptance. The matrix is not a named-agent comparison and does not assign proxy results to Codex Security, KubeIntellect, or Kyverno. It asks a narrower acceptance-boundary question on existing verified records: what would be admitted if the verifier stopped at weaker recorded evidence? For deterministic rules, the domain policy and recorded API-admission flags both pass on 4,989/5,000 records, but the full-verifier snapshot accepts 4,677. For Grok/xAI patches, policy-only acceptance would admit 4,977/5,000 records, while adding the recorded API-admission flag drops acceptance to the full-verifier result of 4,426/5,000 because malformed or dry-run-rejected patches overlap the remaining safety failures. The earlier 19-sample no-policy pilot still records four concrete capability-hardening escapes (`ids 007, 012, 013, 016`); the 5k matrix is larger snapshot-derived corroboration that weaker recorded boundaries trade higher apparent acceptance for records the corresponding full-verifier snapshots rejected. A like-for-like run against a named agent on identical manifests remains the future-work item in Section 9.

6.2 Metrics and Measurement

We measure effectiveness with auto-fix rate, no-new-violations rate, patch size, time-to-patch, risk reduction, and queue position. Auto-fix rate is verifier-accepted patches divided by detected violations; it does not mean automatic production deployment. No-new-violations rate is accepted

Table 4
Acceptance summary; latency reported only for regenerated rows (seed 1337).

Corpus (mode)	Seed	Acceptance	Median proposer (ms)	Median verifier (ms)	Verifier P95 (ms)
Supported (rules, 1,264 records)	1337	1264/1264 (100.00%; 95% CI 99.70–100)	29.0	242.0	517.8
Full corpus (rules+guardrails, 15,718 detections)	1337	13589/13656 (99.51%; 95% CI 99.38–99.61; auto-fix 0.8646 over detections)	–	–	–
Detection slice (Grok/xAI, 1,313 records)	1337	1313/1313 (100.00%; 95% CI 99.71–100)	–	–	–
Grok-5k (Grok/xAI)	1337	4426/5000 (88.52%; 95% CI 87.61–89.37)	–	–	–

Notes: Detection-record counts, Wilson intervals, multi-seed supported-corpus replays, and significance tests are mapped in `ARTIFACTS.md`. The intervals treat records as observations and do not adjust for repeated YAML documents. Dashes are intentionally outside the latency summary: row-level latency was not regenerated for those acceptance artifacts, and the archived Grok 200-record latency telemetry is reported separately in Section 5. Grok server round-trips differ sharply from deterministic verifier latency ($p = 3.2 \times 10^{-47}$).

Table 5
Top 10 API-backed LLM dry-run failure causes by count. Latency medians are reported separately from the 200-detection-record trace spanning 56 unique manifest paths.

Failure Cause	Count
Batch API lookup returned no current object	65
Core/v1 API lookup returned no current object	20
ReplicationController template validation failed	18
Deployment server-side dry-run rejected the object	16
JSON Patch attempted to remove missing `clusterName`	16
ReplicaSet server-side dry-run rejected the object	14
StatefulSet container volume context was invalid	14
Deployment container spec validation failed	12
CronJob job-template validation failed (Keras workload)	11
CronJob job-template validation failed (RetinaNet workload)	11

Table 6
Rejected-record failure categories in two historical verifier snapshots.

Failure category	Full rules historical	Supported historical
can't remove a non-existent object 'clusterName'	58	0
capabilities not defined	0	5
container image missing or empty	0	8
capabilities.drop missing	0	6
privileged container detected	0	6
capabilities.add still contains NET_ADMIN, NET_RAW, SYS_ADMIN	0	4
no containers found in manifest	4	0
manifest lacks required 'spec' member	3	0
capabilities.add still contains SYS_ADMIN	0	2
can't replace a non-existent object 'generateName'	1	0
manifest lacks required 'metadata' member	1	0

patches with zero new policy, API-admission, or universal-safety violations divided by accepted patches; patch size is the median number of JSON Patch operations. Risk reduction uses $\Delta R_i = R_i^{\text{pre}} - R_i^{\text{post}}$. The scheduler replay reports top-50 rank and the wait implied by its configured service-time prior. It does not report open-arrival starvation or empirical operator latency.

Table 7
Acceptance-boundary matrix on checked-in 5k verifier-record snapshots. Escapes are records accepted by a weaker boundary but rejected by the corresponding full-verifier snapshot; the API-admission column uses the recorded `ok_schema` server-side dry-run flag. The rules snapshot is historical and not a fresh rerun of the current verifier.

Dataset	Boundary	Accepted	Accept.	Escapes
rules_5k	Ungated proposer output	5000/5000	100.00%	323
rules_5k	Scanner/policy re-check only	4989/5000	99.78%	312
rules_5k	Policy + API-admission gates	4989/5000	99.78%	312
rules_5k	Full verifier boundary	4677/5000	93.54%	0
grok_5k	Ungated proposer output	5000/5000	100.00%	574
grok_5k	Scanner/policy re-check only	4977/5000	99.54%	551
grok_5k	Policy + API-admission gates	4426/5000	88.52%	0
grok_5k	Full verifier boundary	4426/5000	88.52%	0

Full-run summary. Running the full corpus of 15,718 detections in rules+guardrails mode yields 13,589 accepted out of 13,656 patched items (99.51%; auto-fix rate 0.8646 over detections) with a median of 8 JSON Patch operations and 67 rejected records. In the matched static queue replay, risk-priority ordering gives the top-50 items P95 wait of 7.8 h while FIFO defers the same cohort to 147.2 h under the shared 10-minute service-time fallback.

Targets (Acceptance Criteria). We require auto-fix rate $\geq 70\%$, no-new-violations rate $\geq 95\%$ under the full verifier, and median JSON Patch operations ≤ 6 on curated rules-mode smoke sweeps (median 5, P95 6). Detector hold-out results are wrapper regression coverage only; ArtifactHub structural labels provide bounded agreement evidence. The full-corpus replay has median patch size 8 because multi-violation manifests receive several independent hardening edits, and the weaker-boundary rows in Table 7 intentionally violate the no-new-violations target.

7 Related Work

Legend (Table 8). The evaluated scheduler is a static ordering heuristic, not an online bandit; "guardrails" are policy re-check, patch-application validation, API dry-run, and universal safety checks plus sanitization; "JSON Patch" is a surgical YAML edit; and custom-resource-definition (CRD) fixtures are supporting Kubernetes objects some admission tools require.

Table 8
Lifecycle comparison of Kubernetes remediation systems (May 2026 source snapshot).

Capability	k8s-auto-fix (this work)	LLMSecConfig [19]	Kubecurity [20]	Kyverno [26]
Primary Goal	Proposer-independent acceptance evidence for post-hoc repair	LLM repair guided by static-analysis findings	Context-grounded LLM correction plus schema-guided repair	Policy validation and mutation
Fix Mode	RFC 6902 JSON Patch (rules + optional LLM)	LLM-generated configuration edits	LLM correction plus deterministic schema enforcement	Strategic-merge, RFC 6902, and policy mutation
Acceptance Evidence	Policy re-check + patch application + optional/required-by-campaign API dry-run + explicit safety checks	YAML validation and iterative static-analysis re-scan	Official Kubernetes schema enforcement and newly introduced-misconfiguration measurement	Admission/background-controller policy outcome
Work Ordering	Matched static risk-priority versus FIFO replay	Not evaluated	Not evaluated	No comparable risk-queue evaluation in cited docs
Evaluation Corpus	15,718 detections; 1,000 live-cluster manifests; matched 5,000-record snapshots	1,000 real-world Kubernetes configurations; 94.3% reported success	2,662-issue taxonomy corpus; hybrid repair reports 98.50% correction accuracy	Policy examples and controller tests; no directly comparable public repair corpus in cited docs
Reported Evidence	Per-record outcomes, latency, failure taxonomy, source-mapped figures	Success and newly introduced-misconfiguration rates	Correction, parsing-failure, and newly introduced-misconfiguration rates	Policy/controller outcomes
Outstanding Gaps	Full semantic equivalence and live operator study	Kubernetes server admission and explicit cross-policy invariant gate	No reported RFC 6902 evidence contract combining Kubernetes server-side admission with the four gates	Offline per-patch evidence bundle and risk-queue evaluation

Table 9
Per-policy coverage on shared security-context slices. MAP denotes Kubernetes MutatingAdmissionPolicy; n/a means unavailable or not applicable. Denominators differ, so this is coverage evidence rather than a leaderboard.

Policy	k8s-auto-fix	Kyverno	Polaris (CLI)	Polaris (webhook)	MAP	LLMSecConfig
drop_cap_sys_admin	2/3 (66.7%)	n/a	n/a	n/a	1/3 (33.3%)	0/3 (0.0%)
drop_capabilities	1/2 (50.0%)	12/12 (100.0%)	0/12 (0.0%)	0/12 (0.0%)	1/2 (50.0%)	0/4 (0.0%)
env_var_secret	n/a	3/3 (100.0%)	0/3 (0.0%)	0/3 (0.0%)	n/a	n/a
no_host_path	1/1 (100.0%)	n/a	n/a	n/a	0/1 (0.0%)	0/1 (0.0%)
no_host_ports	0/1 (0.0%)	n/a	n/a	n/a	0/1 (0.0%)	0/1 (0.0%)
no_latest_tag	1/2 (50.0%)	25/25 (100.0%)	0/25 (0.0%)	0/25 (0.0%)	1/2 (50.0%)	0/2 (0.0%)
no_privileged	1/3 (33.3%)	5/5 (100.0%)	0/5 (0.0%)	0/5 (0.0%)	0/1 (0.0%)	0/1 (0.0%)
read_only_root_fs	2/3 (66.7%)	107/107 (100.0%)	0/107 (0.0%)	86/107 (80.4%)	1/3 (33.3%)	0/3 (0.0%)
run_as_non_root	2/3 (66.7%)	63/63 (100.0%)	0/63 (0.0%)	37/63 (58.7%)	1/3 (33.3%)	0/3 (0.0%)
set_requests_limits	4/6 (66.7%)	285/285 (100.0%)	0/285 (0.0%)	135/285 (47.4%)	1/3 (33.3%)	0/6 (0.0%)

Metric caveat. Table 8 reports each system's own unit and protocol; the percentages are not directly comparable and provide lifecycle context only.

Table 9 grounds those differences with shared per-policy slices, not a single aggregate leaderboard. Tables 8 and 9 therefore use different rosters on purpose: Table 8 compares the closest repair/validation contracts, while Table 9 is limited to tools with reusable policy-level artifacts or scanner interfaces for the same slices. `k8s-auto-fix` lands 33–100% acceptance across targeted high-risk policies, while unsupported `no_host_ports` remains 0%. Some cells have denominators as small as one, and Kyverno/Polaris rows use larger detection sets; we therefore treat the table only as coverage evidence. Kyverno's mutate CLI reaches 100% on overlapping checks when fixtures are present, but admission can fail before mutation when they are absent. Polaris is detection-oriented, Kubernetes MutatingAdmissionPolicy (MAP) depends on the Common Expression Language/JSONPatch admission-policy surface [4], and the tiny aligned LLMSecConfig slice only illustrates compatibility with this

verifier contract—it does not establish system-level inferiority.

We group prior work into five categories—detection-only tooling, admission-time policy engines, LLM-based configuration repair, large-scale SRE automation, and emerging security agents—and state, for each, the gap that motivates a closed verification loop. Table 8 summarizes the comparison; the discussion below makes the categorization explicit.

1. Detection-only tooling. Static analyzers such as `kube-linter` [6] and dashboards such as Polaris [7] identify misconfigurations and score manifests against best-practice checks; GLITCH shows the broader value of technology-agnostic IaC security-smell detection [34]. Their output, however, is a finding rather than a validated, ready-to-apply patch. We reuse such detectors as the *Detector* stage and add the missing repair-and-verify stages on top.

2. Admission-time policy engines. Kubernetes admission control runs mutating logic before validating logic [3]; Kyverno [26] and OPA Gatekeeper [5] validate or mutate resources at admission time. Kyverno additionally supports RFC 6902/strategic-merge mutation and background `mutateExisting` processing for persisted resources [27]. Our distinction is therefore not the ability to alter an existing object. It is the production of an offline RFC 6902 candidate accompanied by campaign-recorded policy, patch-application, API-admission, and safety evidence before GitOps review or application.

3. LLM-based configuration repair. GenKubeSec [24] detects, localizes, explains, and suggests remediations for Kubernetes misconfigurations. Minna *et al.* mine Artifact Hub Helm charts, compare Checkov/KICS findings, and evaluate LLM-generated mitigations on the same ecosystem our corpus draws from [22]; Lian *et al.* study LLM-based configuration validation across systems [23]. LLMSecConfig [19] already implements an iterative detect–repair–validate–re-scan loop and reports 94.3% success over 1,000 real-world Kubernetes configurations. Kubecurity [20] combines context-grounded LLM correction with official Kubernetes JSON-schema enforcement and reports 98.50% correction accuracy with fewer newly introduced misconfigurations. KubeGuard [21] uses manifests and runtime logs to create or refine least-privilege Roles, NetworkPolicies, and Deployments, presenting the resulting manifests for operator review. These are the closest adjacent repair loops. The narrower distinction studied here is a common RFC 6902 candidate/evidence contract that adds patch applicability, Kubernetes server-side admission in live campaigns, and explicit cross-policy invariants.

4. Large-scale SRE automation. Production fleet-management systems such as Borg [28] schedule and manage long-running workloads at scale, but they target fleet operation rather than evidence for accepting a static manifest repair.

5. Emerging security agents. Concurrent with this work, OpenAI describes Codex Security as a research-preview application-security agent [30], while KubeIntellect [31] and KubeLLM [25] propose LLM-orchestrated agents for Kubernetes management and troubleshooting. These agents target general software vulnerabilities or broad cluster operations; neither is centered on enforcing policy re-check, patch-application validation, server-side dry-run, and universal safety as the acceptance boundary for each patch. Their published interfaces and evaluations also target different tasks: Codex Security scans connected GitHub repositories, while KubeIntellect evaluates natural-language Kubernetes-management queries. Neither reports results under our identical-manifest remediation protocol. We therefore position against them qualitatively and leave a matched agent comparison to future work in Section 9.

Positioning. Prior systems already close important detect–repair–validate loops. Our narrower contribution is to make the acceptance boundary explicit and proposer-independent: trigger-policy re-check, RFC 6902 applicability, Kubernetes server-side admission, and cross-policy safety invariants, with every released campaign labeled by the gates it actually required. This emphasis follows a broader automated-program-repair lesson: scanner- or test-passing patches can still be overfit or unsafe unless the acceptance boundary checks the domain property the patch was meant to preserve [35]–[38].

Static analysis and API-level validation. Many tools scan manifests offline. In live campaigns, our verifier additionally exercises the Kubernetes API with `kubectl apply --dry-run=server` [10], submitting the request without persistence so cluster-specific problems—missing CRDs, namespaces, or quotas—are caught before apply. We call this API-level admission validation rather than dynamic application-security testing.

8 Limitations

The system prioritizes verifier evidence over autonomous deployment, so several constraints remain:

- **External validity:** the corpora skew toward Helm-derived workloads; we refresh the ArtifactHub scrape monthly, add partner manifests as available, and will use the drafted operator survey to supplement deterministic replays.
- **Detector validity:** the wrapper delegates to `kube-linter` and policy-engine findings, and the separately implemented ArtifactHub structural oracle is a single-annotator, Helm-rendered slice rather than a broad detector benchmark.
- **Fixture sensitivity:** server dry-runs need namespaces, service accounts, CRDs, and related objects that mirror production; the fixture harness installs common dependencies and records misses for future cluster-specific bundles.
- **LLM/provider risk:** LLM-backed patches carry latency, cost, provider-drift, and hallucinated-field risks, so every LLM patch is cached and telemetry-backed. Its acceptance evidence is limited to the verifier configuration recorded for that campaign.
- **Verification scope:** the verifier implements policy re-check, patch-application validation, API admission, and universal safety checks, but historical offline campaigns did not all require a live API gate. No campaign establishes full workload semantic equivalence; snapshot-derived ablations inherit the veri-

Table 10
Evidence status. We distinguish completed campaigns from deterministic replays and planned work.

Evidence	Status	Scope
Rules pipeline	Completed	Deterministic records (1,264 supported; 15,718-detection full run)
Grok/xAI run	Completed (opt-in)	Credentialed external API; 5,000-detection-record sweep
Live replay	Completed	Fixture-seeded single cluster, 1,000 manifests
Cross-cloud replay	Completed (replay outputs)	200 manifests each on EKS/GKE/AKS-labeled targets; provider environment recorded in the artifact, not independently re-verified here
Scheduler comparison	Replay-based	Matched finite static queue replay, not a live operator deployment or open-arrival fairness study
Human-in-the-loop rotation	Planned	Survey instrument drafted; future work

fier version and configuration that generated each record.

- **Scheduler validity:** the replay uses a historical supported snapshot, historical pass priors, a configured 10-minute service-time fallback for every policy, zero initial ages, and zero exploration input. It demonstrates static prioritization only; it does not demonstrate online learning, an aging effect, empirical operator latency, or starvation prevention under arrivals.
- **Human-review boundary:** high-risk semantic edits such as non-allowlisted `hostPath` rewrites, dropped capabilities, or read-only-root-filesystem changes with unknown runtime needs should route through human review unless the workload requirement is known.
- **Experimental validity:** scheduler and cross-cloud results are deterministic replays. The published Codex Security and KubeIntellect evaluations use different task interfaces, so no like-for-like agent comparison was completed.

9 Conclusion and Future Work

The closed-loop verifier records 100.0% server-side dry-run and live apply acceptance on a fixture-seeded live-cluster replay (1,000/1,000 stratified manifests; Tables 4 and 10, data/live_cluster/results_1k.json). Offline, the supported 1,264-record slice records 100.00% acceptance in rules mode, the 1,313-record Grok/xAI slice records 100.00%, Grok-5k records 88.52%, and rules+guardrails accept 13,589/13,656 patched items (99.51%; auto-fix 0.8646 over 15,718 detections) with median patch size 8 (Table 4, data/metrics_rules_full.json). The offline rows report record-level outcomes under their stored verifier configurations, not manifest-level or universal API-admission rates. In the matched finite queue replay, static risk priority trims top-50 P95 wait from 147.2 h (FIFO) to 7.8 h under the shared 10-minute service-time fallback (data/metrics_schedule_compare.json). These results support checked remediation and static threat prioritization; they do not establish online bandit learning or starvation prevention.

The summary tables regenerate from checked-in JSON/CSV artifacts by `make reproducible-report` (this target re-renders from checked-in artifacts and does not re-execute proposers or verifiers); live replay, cross-cluster replay, LLM API reruns, and figures are mapped in `ARTIFACTS.md`. Scheduler comparisons stem from deterministic queue replays rather than live analyst rotations. The evidence is anchored in deterministic guardrails and campaign-specific verifier records, with server-side API-admission and live-apply claims restricted to the fixture-seeded replay. Matching xAI/Grok reruns require credentials and budget; their cost can be recomputed from published token counts and current pricing [11]. Every accepted patch is surfaced through a GitOps helper that records verifier evidence, captures the JSON Patch diff, and requires human approval before merge [8], [12]. Auto-apply should stay off for image substitutions, required `hostPath` paths, read-only-root-filesystem or dropped-capability changes with unknown runtime needs, and regulated namespaces; those cases route to review.

Future work will fold EPSS and CISA KEV feeds into R_i, implement and evaluate an explicit sequential policy with feedback under open arrivals, and conduct a live human-in-the-loop rotation. We will also broaden seeded fixtures and Kyverno webhook baselines across new corpora, then design adapters and a shared acceptance protocol for matched agent comparisons. KubeIntellect is the near-term public candidate; a Codex Security comparison would also need to map its repository-connected workflow to the identical-manifest protocol without conflating different tasks. That experiment would connect Kubernetes remediation to closed-loop repair benchmarks for general software patches [18].

Artifact availability. The complete artifact—source, evaluation scripts, datasets, telemetry, and the `make reproducible-report` entry point—is public at commit `06745966782efd479aada2127e745c8d29368ea9` (release tag `tcc-2025-12-0666-evidence-2026-07-16`). All reported metrics were generated with random seed `1337` on Python 3.12.4 (`kubectl` 1.34.1, `kube-linter` 0.7.6, `kind` 0.30.0; Kubernetes 1.34.0). The optional Grok/xAI proposer requires external credentials and is cached so that artifact evaluation does not depend on live API access; checked-in JSON/CSV data and generated figures support the reported tables and figures without rerunning that API path.

References

[1] CIS Kubernetes Benchmarks. Accessed: May 2026. [Online]. Available: https://www.cisecurity.org/benchmark/kubernetes

[2] Kubernetes: Pod Security Standards. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/concepts/security/pod-security-standards/

[3] Kubernetes: Admission Control. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/

[4] Kubernetes: MutatingAdmissionPolicy. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/reference/access-authn-authz/mutating-admission-policy/

[5] OPA Gatekeeper How-to. Accessed: May 2026. [Online]. Available: https://open-policy-agent.github.io/gatekeeper/website/docs/howto/

[6] *kube-linter* Documentation. Accessed: May 2026. [Online]. Available: https://docs.kubelinter.io/

[7] Fairwinds, "Polaris: Validation of best practices in your Kubernetes clusters," GitHub repository. Accessed: May 2026. [Online]. Available: https://github.com/FairwindsOps/polaris

[8] Kubernetes: Configure a Security Context. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/security-context/

[9] RFC 6902: JSON Patch, DOI:10.17487/RFC6902. Accessed: May 2026. [Online]. Available: https://www.rfc-editor.org/info/rfc6902

[10] `kubectl apply` Command Reference (server-side dry-run). Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_apply/

[11] xAI, "API Pricing." Accessed: May 2026. [Online]. Available: https://docs.x.ai/developers/pricing

[12] Kubernetes: Seccomp and Kubernetes. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/reference/node/seccomp/

[13] NIST National Vulnerability Database (NVD). Accessed: May 2026. [Online]. Available: https://nvd.nist.gov/

[14] CISA Known Exploited Vulnerabilities Catalog. Accessed: May 2026. [Online]. Available: https://www.cisa.gov/known-exploited-vulnerabilities-catalog

[15] FIRST Exploit Prediction Scoring System (EPSS). Accessed: May 2026. [Online]. Available: https://www.first.org/epss/

[16] Trivy: Vulnerability Scanner for Containers and IaC. Accessed: May 2026. [Online]. Available: https://aquasecurity.github.io/trivy/

[17] Grype: A Vulnerability Scanner for Container Images and Filesystems. Accessed: May 2026. [Online]. Available: https://github.com/anchore/grype

[18] SWE-bench Verified (background on closed-loop code repair evaluation). Accessed: May 2026. [Online]. Available: https://openai.com/index/introducing-swe-bench-verified/

[19] Z. Ye, T. H. M. Le, and M. A. Babar, "LLMSecConfig: An LLM-Based Approach for Fixing Software Container Misconfigurations," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025.

[20] M. A. Ghorab, A. A. Latif, and M. A. Saied, "Kubernetes Misconfigurations in the Wild: Taxonomy, Evolution, and Automated Repair with Large Language Models," in *Proc. IEEE/ACM Int. Conf. on AI Foundation Models and Software Engineering (AIware)*, 2026. [Online]. Available: https://openreview.net/forum?id=dTziAt4nLv

[21] O. Sgan Cohen, E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "KubeGuard: LLM-Assisted Kubernetes Hardening via Configuration Files and Runtime Logs Analysis," *arXiv preprint arXiv:2509.04191*, 2025.

[22] F. Minna, F. Massacci, and K. Tuma, "Analyzing and Mitigating (with LLMs) the Security Misconfigurations of Helm Charts from Artifact Hub," *Empirical Software Engineering*, vol. 30, article 132, 2025, doi: 10.1007/s10664-025-10688-0.

[23] X. Lian, Y. Chen, R. Cheng, J. Huang, P. Thakkar, M. Zhang, and T. Xu, "Configuration Validation with Large Language Models," *arXiv preprint arXiv:2310.09690*, 2023.

[24] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation," *arXiv preprint arXiv:2405.19954*, 2024.

[25] M. De Jesus, P. Sylvester, W. Clifford, A. Perez, and P. Lama, "LLM-Based Multi-Agent Framework For Troubleshooting Distributed Systems," in *Proc. 2025 IEEE Cloud Summit*, 2025, pp. 110–115, doi: 10.1109/Cloud-Summit64795.2025.00024.

[26] Kyverno Project, "Kyverno Documentation," Accessed: May 2026. [Online]. Available: https://kyverno.io/docs/

[27] Kyverno Project, "Mutate Rules," Accessed: May 2026. [Online]. Available: https://kyverno.io/docs/policy-types/cluster-policy/mutate/

[28] A. Verma *et al.*, "Large-scale Cluster Management at Google with Borg," in *Proc. EuroSys*, 2015; supplemental SRE updates accessed May 2026. [Online]. Available: https://research.google/pubs/pub43438/

[29] ArtifactHub Documentation. Accessed: May 2026. [Online]. Available: https://artifacthub.io/docs/

[30] OpenAI, "Codex Security: now in research preview," 2026. [Online]. Available: https://openai.com/index/codex-security-now-in-research-preview/

[31] M. S. Ardebili and A. Bartolini, "KubeIntellect: A Modular LLM-Orchestrated Agent Framework for End-to-End Kubernetes Management," *arXiv preprint arXiv:2509.02449*, 2025.

[32] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks," *arXiv preprint arXiv:2312.12575*, 2024.

[33] M. S. Islam Shamim, F. A. Bhuiyan, and A. Rahman, "XI Commandments of Kubernetes Security: A Systematization of Knowledge Related to Kubernetes Security Practices," in *Proc. 2020 IEEE Secure Development Conf. (SecDev)*, 2020, pp. 58–64, doi: 10.1109/SecDev45635.2020.00025.

[34] N. Saavedra and J. F. Ferreira, "GLITCH: Automated Polyglot Security Smell Detection in Infrastructure as Code," in *Proc. 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022, article 47, 12 pages, doi: 10.1145/3551349.3556945.

[35] C. S. Xia, Y. Wei, and L. Zhang, "Automated Program Repair in the Era of Large Pre-trained Language Models," in *Proc. 2023 IEEE/ACM 45th Int. Conf. on Software Engineering (ICSE)*, 2023, pp. 1482–1494, doi: 10.1109/ICSE48619.2023.00129.

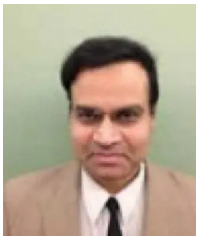
[36] J. Petke, M. Martinez, M. Kechagia, A. Aleti, and F. Sarro, "The Patch Overfitting Problem in Automated Program Repair: Practical Magnitude and a Baseline for Realistic Benchmarking," in *Companion Proc. 32nd ACM Int. Conf. on the Foundations of Software Engineering (FSE Companion)*, 2024, pp. 452–456, doi: 10.1145/3663529.3663776.

[37] W. Yang, L. Song, and Y. Xue, "Rust-lancet: Automated Ownership-Rule-Violation Fixing with Behavior Preservation," in *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2024, pp. 1034–1046, doi: 10.1145/3597503.3639103.

[38] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," in *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2025, doi: 10.1109/ICSE55347.2025.00157.

[39] R. Shu, X. Gu, and W. Enck, "A Study of Security Vulnerabilities on Docker Hub," in *Proc. 7th ACM Conf. on Data and Application Security and Privacy (CODASPY)*, 2017, pp. 269–280, doi: 10.1145/3029806.3029832.

Brian Mendonca is an M.S. student at the Georgia Institute of Technology (2024–2026) focusing on secure DevOps, policy-driven remediation, and developer tooling. He received the B.E. in Mechanical Engineering (summa cum laude, GPA 3.99) from Arizona State University in 2021. His prior engineering roles at BAE Systems, Tube Specialties Inc., and BD span aerospace and biomedical quality, Lean/Six Sigma improvement, CAPA, and risk-based quality systems. His research interests include secure configuration management for cloud-native systems, infrastructure-as-code analysis, and data-informed quality engineering.



Vijay K. Madisetti is Professor of Cybersecurity and Privacy at the Georgia Institute of Technology. He earned his Ph.D. in Electrical Engineering and Computer Sciences from the University of California at Berkeley. Professor Madisetti is a Fellow of the IEEE and a recipient of the ASEE Terman Medal. He has authored widely referenced textbooks on cloud computing, data analytics, blockchain, and microservices, and has extensive experience in secure system architectures and privacy-preserving technologies.